AWS SERIES · CONTAINERS · EKS ARCHITECTURE

Amazon EKS Architecture: The Big Picture

A beginner's guide to how Amazon EKS works: what containers and Kubernetes are, which parts AWS runs and which run in your account, and what happens step by step when you deploy an app, when a user calls it, and when a server fails.

PA

Tutor: Prudhvi Akella
Lead Data & AI Engineer · 13+ years in data engineering and AI architecture

Services: Amazon EKS · Kubernetes · Amazon ECR · EC2 / EKS Auto Mode · VPC · Load balancer · IAM

Before this module: Module 1 (Networking) and Module 2 (IAM). Sample: AcmeCorp's HR assistant on EKS in `us-east-1`. Concepts only — no lab

1. Containers in 2 minutes
2. Why containers need an orchestrator
3. Kubernetes in one idea: desired state
4. Master and worker architecture
5. What EKS is
6. The control plane
7. A worker node
8. Three ways to get worker nodes
9. Kubernetes objects you will meet
10. Every pod gets an IP
11. Getting user traffic in
12. Permissions: two questions
13. Scaling
14. Flow 1: what happens when you deploy
15. Flow 2: one request to the HR assistant
16. Flow 3: a server fails
17. The full picture
18. EKS, ECS or Lambda?
19. What EKS costs
20. What you can skip for now
21. Words to know
22. Key takeaways

1 Containers in 2 minutes

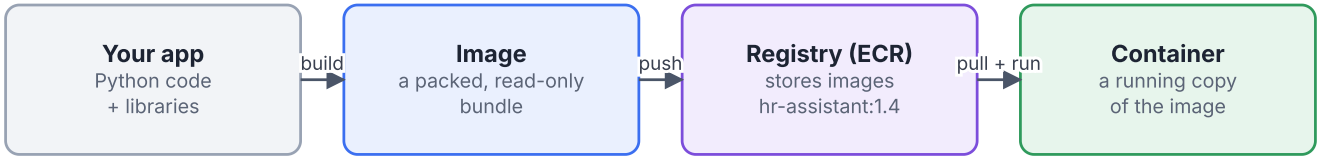


Figure 1. Code + everything it needs → an image → stored in a registry → run anywhere as a container.

Word	Meaning	HR assistant
Image	Your code plus everything it needs (Python, libraries, settings), packed into one file	`hr-assistant:1.4`
Container	A running copy of an image. Starts in seconds, runs the same on any machine	The HR assistant answering questions

Word	Meaning	HR assistant
Registry	A store for images. On AWS: **Amazon ECR**	`…dkr.ecr.us-east-1.amazonaws.com/hr-assistant`

Why containers? "It works on my laptop" becomes "it works everywhere" — the same image runs on your laptop, on a test server and in production.

2 Why containers need an orchestrator

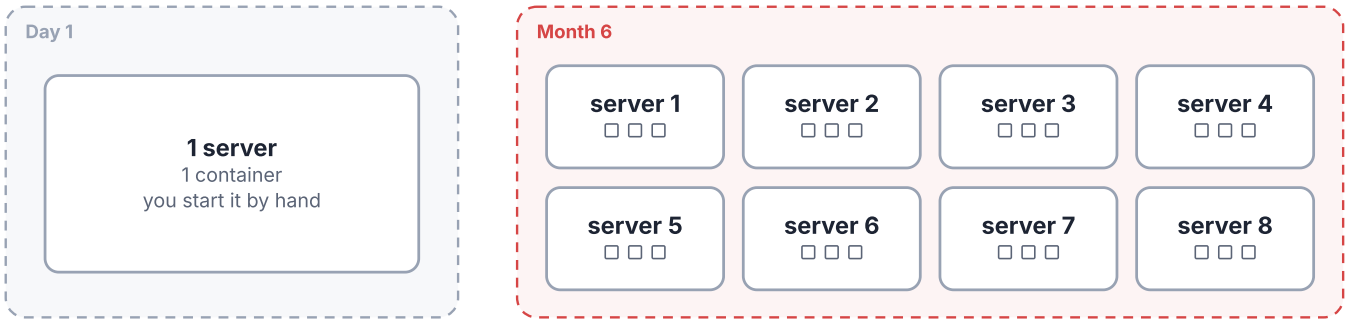


Figure 2. One container on one server is easy. Dozens of containers on many servers need something to manage them.

Question	Who answers it
Which server should this container run on?	The orchestrator picks one with free room
A container crashed at 3 a.m. — who restarts it?	The orchestrator, automatically
A whole server died — what about its containers?	The orchestrator starts them on other servers
Traffic doubled — how do we run more copies?	Change one number, or let it scale by itself
How do users reach containers that keep moving?	One stable address in front of them

A tool that does all this is a **container orchestrator**. **Kubernetes** is the most used one. **Amazon EKS** is Kubernetes, run for you by AWS.

3 Kubernetes in one idea: desired state

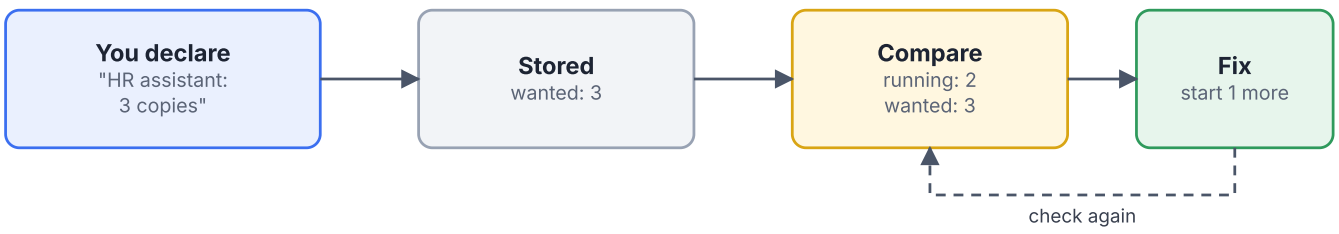


Figure 3. You write what you want. Kubernetes keeps comparing it with what is running and fixes the difference.

You do not tell Kubernetes "start a container". You tell it "I want 3 copies of this app running". This is called the **desired state**. Kubernetes then repeats a simple loop, all day:

1. **Read what you want:** 3 copies.
2. **Look at what is running:** maybe 2, because one crashed.
3. **Fix the difference:** start 1 more.
4. **Repeat** every few seconds.

> Restarting, scaling and self-healing are all the same idea: make what is running match what you asked for.

4 Master and worker architecture

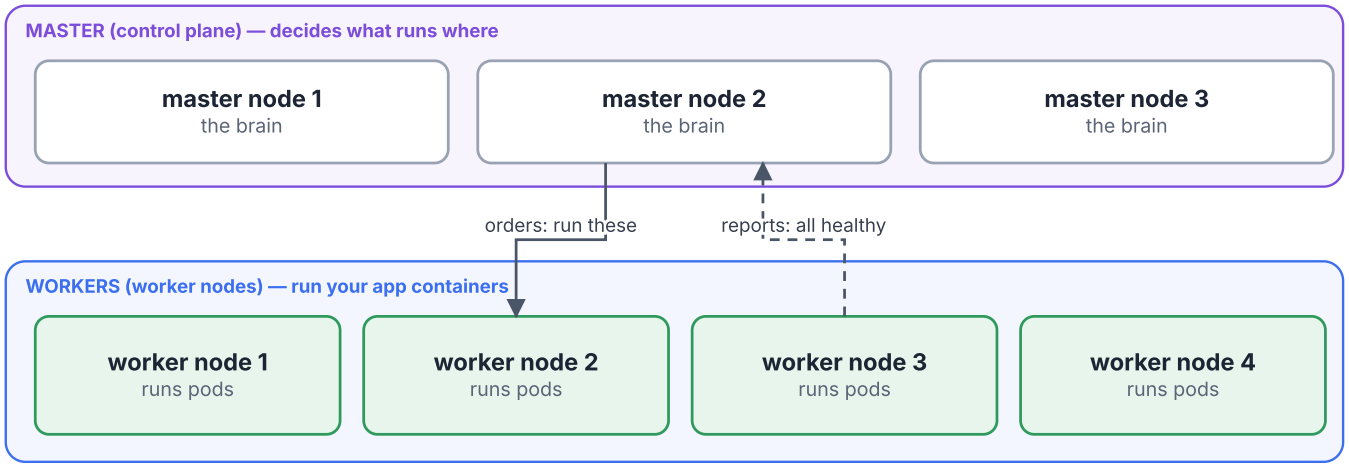


Figure 4. The master–worker pattern. Masters decide; workers run the apps and report back.

Kubernetes uses the classic **master–worker** pattern (older material calls it **master–slave**). A few **master** machines make every decision. Many **worker** machines do the actual work.

Role	Does	Does not
Master	Receives your requests, remembers what you want, decides which worker runs each container, reacts when something breaks	Run your app
Worker	Runs the containers it is told to run, and reports their health back	Decide anything on its own

You may read	Today's name	On EKS
Master, master node	**Control plane**	Run by AWS — you never see these machines
Slave, minion, node	**Worker node**	Servers in your account, or managed by AWS

- Kubernetes renamed "master" to **control plane**. Use the new names; recognise the old ones in tutorials and interviews.
- There are usually **3 masters**, in different AZs, so the cluster keeps working if one fails.

Interview line: "Kubernetes uses a master–worker architecture. The control plane decides and stores the desired state; worker nodes run the pods. In EKS, AWS runs the control plane, so we only look after the workers and our apps."

5 What EKS is

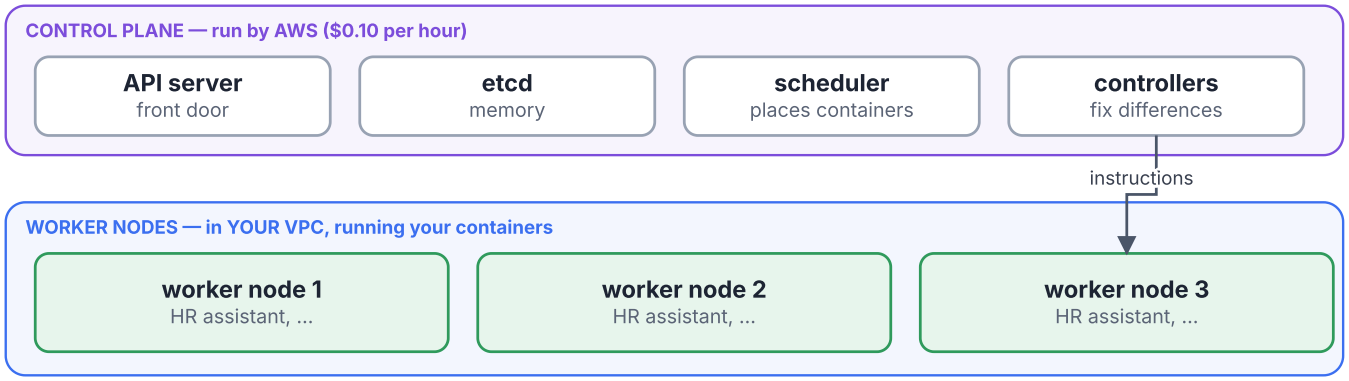


Figure 5. EKS has two halves. AWS runs the brain in its own account. Your containers run on servers in YOUR VPC.

Amazon EKS (Elastic Kubernetes Service) is standard Kubernetes where **AWS runs the control plane** (the masters). Your containers run on worker nodes in your own AWS account.

	AWS takes care of	You take care of
Control plane (masters)	Everything: servers, 3 AZs, patching, backups	Upgrading the Kubernetes version once a year
Worker nodes	Everything, if you choose **Auto Mode**	Sizes and updates, if you manage them yourself
Your app	—	The image, how many copies, its permissions

You never log in to the control plane. You talk to it with the `kubectl` command or the AWS console.

6 The control plane

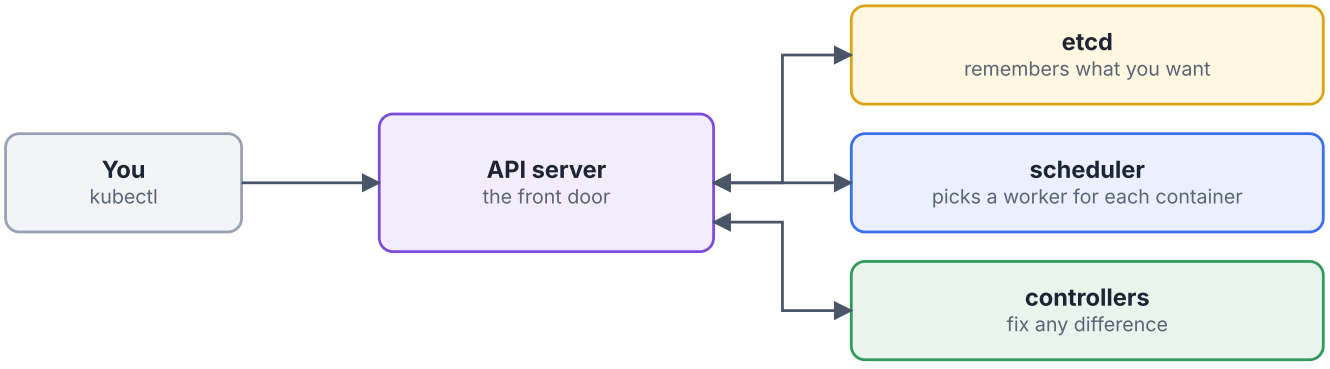


Figure 6. Everything goes through the API server.

Part	Its one job	HR assistant example
API server	The only way in. Checks who you are and saves your request	Receives "run 3 copies of hr-assistant"
etcd	The cluster's memory: everything you asked for	Remembers: hr-assistant, 3 copies
Scheduler	Picks a worker node with free room for each new container	Puts copy 3 on worker node 2
Controllers	Keep checking and fix any difference	Sees 2 running, wants 3 → asks for 1 more

7 A worker node

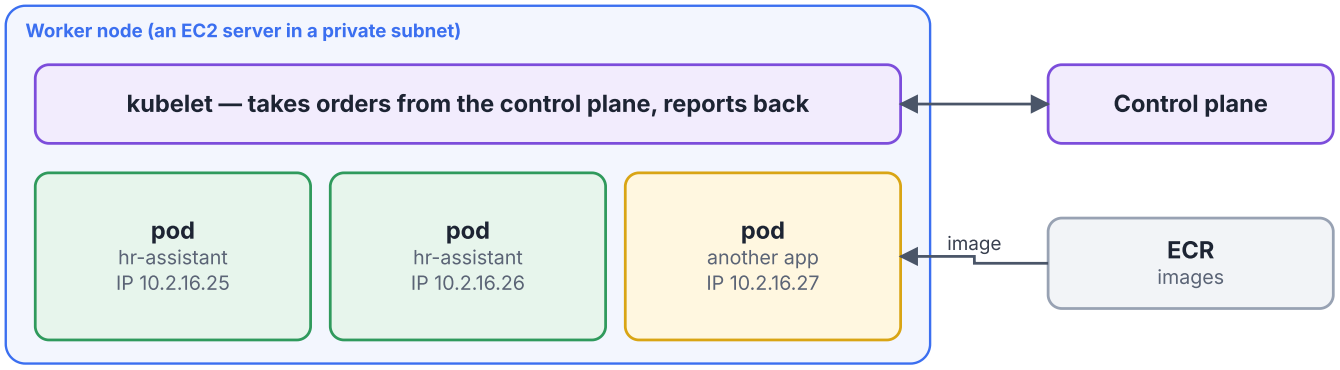


Figure 7. A worker node is a server with one agent (kubelet). Pods are where your containers run.

Part	Job
kubelet	The agent on every worker: receives "run these pods", starts them, reports their health
Pod	The smallest thing Kubernetes runs: your container, with its own IP address

Pods are **replaceable**. When one crashes or is updated, a new pod with a new IP takes its place. That is why users never connect to a pod directly (section 11).

8 Three ways to get worker nodes

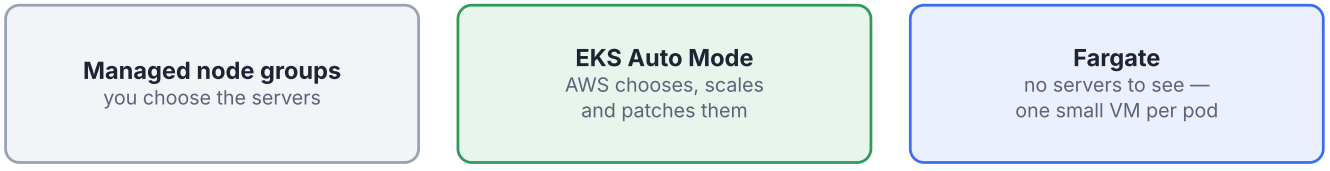


Figure 8. Who looks after the worker servers?

Option	You manage	Good for
Managed node groups	Server type and count, updates	Full control, GPUs, special needs

Option	You manage	Good for
EKS Auto Mode	Almost nothing — just your app	**Most new teams** — recommended
Fargate	Nothing	Small or occasional workloads

> Start with **Auto Mode**. AWS looks after the servers; you look after your app.

9 Kubernetes objects you will meet

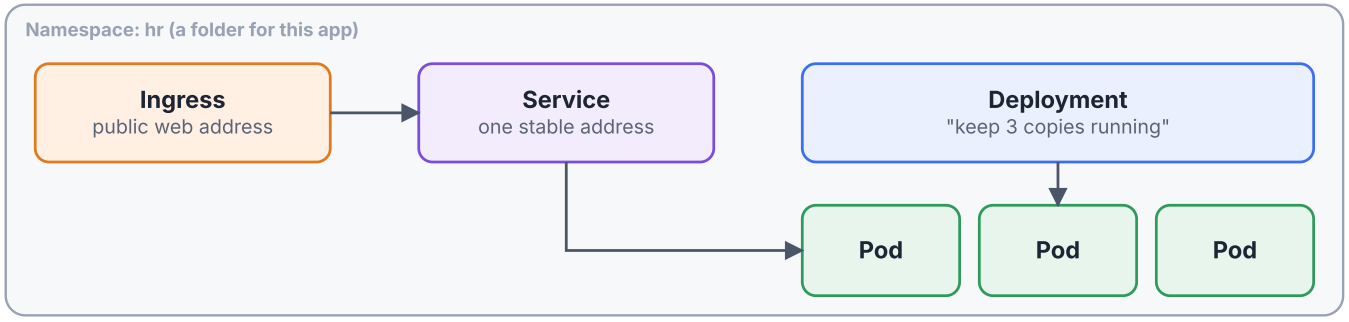


Figure 9. The main objects behind the HR assistant.

Object	What it is	HR assistant
Pod	One running copy of your container	One copy of the app
Deployment	"Keep N copies of this image running"	3 copies of `hr-assistant:1.4`
Service	One stable address in front of changing pods	`hr-assistant` → the 3 pods
Ingress	Brings outside web traffic in (creates a load balancer)	`hr.acmecorp.com`
Namespace	A folder that keeps one app's objects together	`hr`
ConfigMap / Secret	Settings / passwords given to the app	`MODEL_ID`, an API key

You describe objects in a short **YAML** file. This one is the desired state from section 3:

```
kind: Deployment
metadata:
  name: hr-assistant
spec:
  replicas: 3                    # I want 3 copies
  template:
    spec:
      containers:
        - name: api
          image: 111122223333.dkr.ecr.us-east-1.amazonaws.com/hr-assistant:1.4   # which image
```

> The real file has a few more required lines. The idea is just this: **which image, how many copies**.

10 Every pod gets an IP

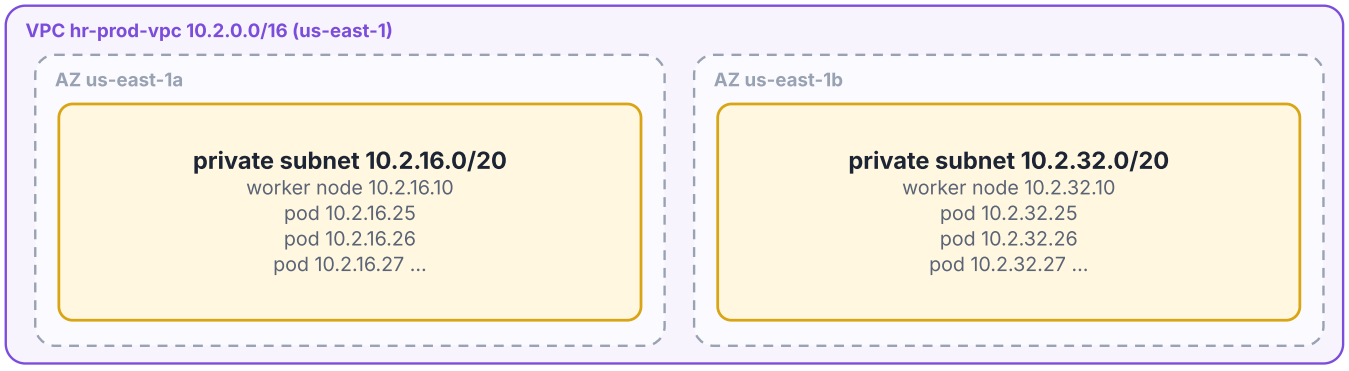


Figure 10. Worker nodes run in private subnets of your VPC. Every pod gets its own IP from the subnet.

An EKS cluster lives in **your VPC** (Module 1). Worker nodes sit in **private subnets**, spread over at least 2 AZs.

Thing	Takes an IP from your subnet?
Worker node	Yes, 1
Every pod	**Yes, 1 each**
Load balancer	Yes, a few

Module 1 again: **more devices → more IPs**. Every pod is a device. 500 pods need 500 free IPs. If the subnet runs out, new pods cannot start — so EKS subnets are large (`/20` or bigger).

11 Getting user traffic in

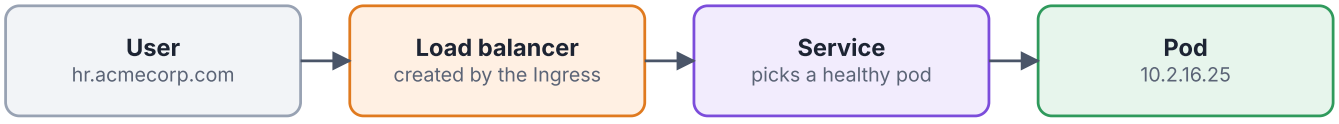


Figure 11. How a user's request reaches a pod.

1. **You create an Ingress** for `hr.acmecorp.com`.
2. **EKS creates an AWS load balancer** in the public subnets.
3. **The load balancer sends each request** to a healthy pod through the **Service**.

Only the load balancer is public. Pods stay in private subnets (Module 1: "only the load balancer is public").

12 Permissions: two questions

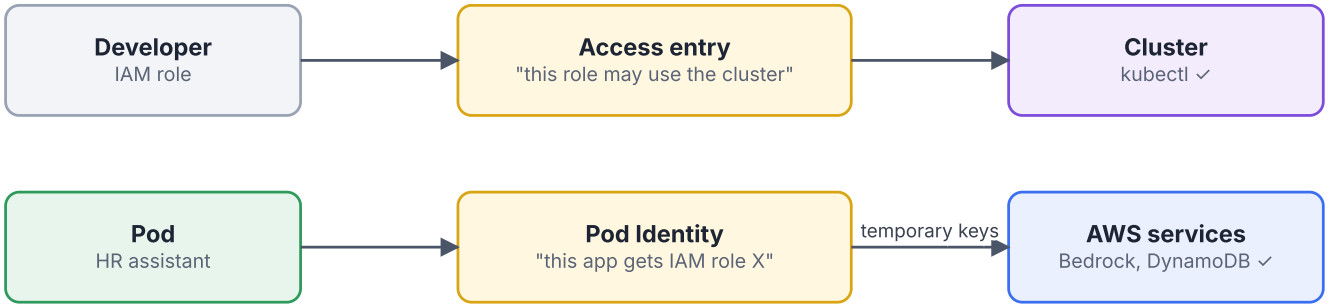


Figure 12. Two separate permission questions.

Question	EKS answer
Who may use the cluster? (people, pipelines)	An **access entry** links an IAM user or role to the cluster, with admin, edit or view rights
What may an app call in AWS? (Bedrock, S3, DynamoDB)	**EKS Pod Identity** gives the app's pods an IAM role. They receive temporary keys automatically

> Never put AWS access keys inside an image or a config file. Pod Identity gives each app only the permissions it needs (Module 2).

13 Scaling



Figure 13. Busy? First more pods, then — if there is no room — more servers.

Level	What grows	Who does it
Pods	More copies of your app	A pod autoscaler (HPA), based on CPU or other measures
Worker nodes	More servers to hold those pods	**Auto Mode** — and it removes empty ones when traffic drops

14 Flow 1: what happens when you deploy

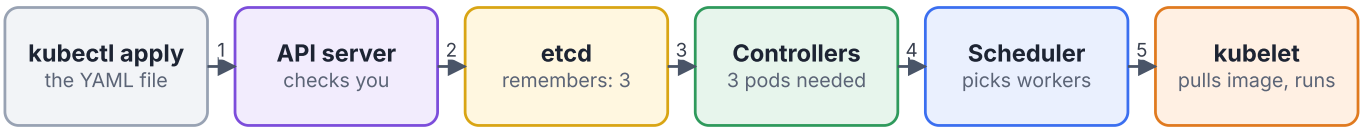


Figure 14. Flow 1 — from "deploy" to a running app. Numbers match the steps.

1. **You run** `kubectl apply` with the YAML file from section 9.
2. **The API server checks who you are** (your access entry) and accepts the request.
3. **etcd remembers it:** "hr-assistant, 3 copies".
4. **A controller sees 0 running, 3 wanted,** and asks for 3 pods.
5. **The scheduler picks a worker node** with free room for each pod.
6. **The kubelet on that worker** pulls the image from ECR and starts the container. The pod gets an IP and is running.

15 Flow 2: one request to the HR assistant

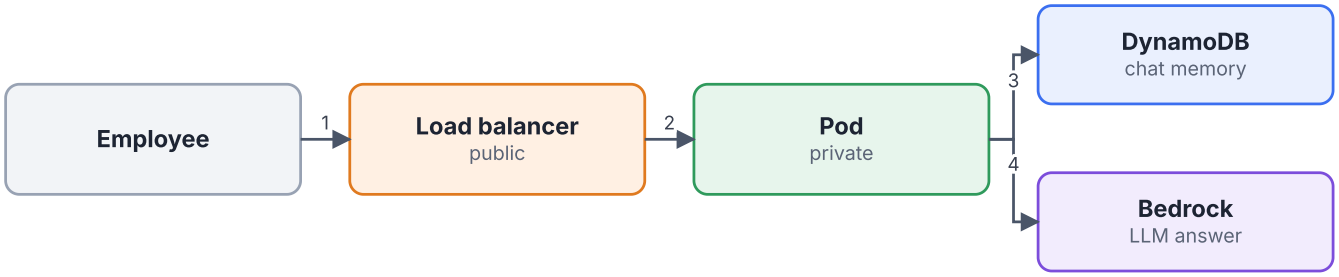


Figure 15. Flow 2 — one question to the HR assistant.

1. **The employee opens** `hr.acmecorp.com` and asks a question.
2. **The load balancer sends it to a healthy pod.**
3. **The app loads the chat history from DynamoDB** using its Pod Identity role.
4. **It calls Bedrock and returns the answer,** then saves the conversation to DynamoDB.

16 Flow 3: a server fails

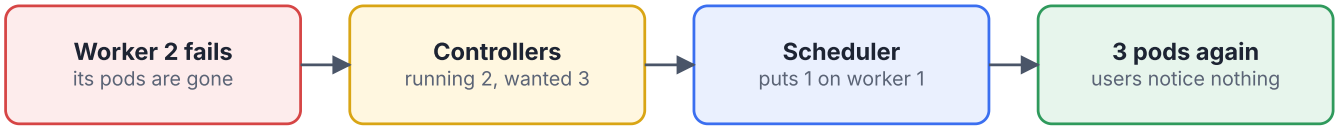


Figure 16. Flow 3 — a server dies, nobody is woken up, the app keeps answering.

1. **Worker node 2 stops.** The pods on it are gone.
2. **The load balancer stops sending users there** and uses the pods on other workers.
3. **Controllers see 2 running, 3 wanted,** and ask for a new pod.
4. **The scheduler places it on another worker** (Auto Mode adds a server if all are full). Back to 3.

> This is desired state (section 3) in action. It is also why you always run **at least 2 copies in 2 AZs**.

17 The full picture

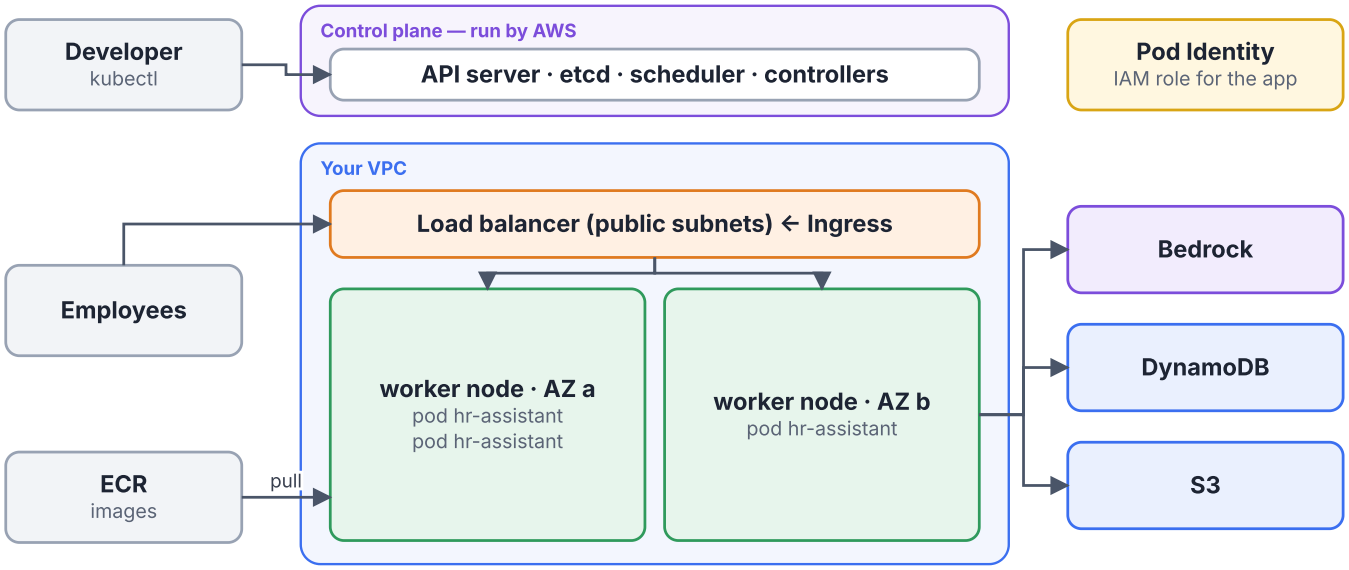


Figure 17. The full picture: AcmeCorp's HR assistant on EKS in us-east-1.

Piece	Where it lives	Section
Control plane (masters)	AWS's account, 3 AZs	4, 5, 6
Worker nodes and pods	Your VPC, private subnets, 2 AZs	7, 8, 10
Deployment, Service, Ingress	Inside the cluster, namespace `hr`	9
Load balancer	Your VPC, public subnets	11
Access entries / Pod Identity	EKS + IAM	12
ECR, Bedrock, DynamoDB, S3	AWS services in us-east-1	1, 15

18 EKS, ECS or Lambda?

	Lambda	ECS	EKS
You run	Functions	Containers (AWS's own system)	Containers (Kubernetes)
How hard to learn	Easiest	Easy	Hardest

	Lambda	ECS	EKS
Long-running apps, streaming	Up to 15 min per call	Yes	Yes
GPUs, your own model servers	No	Limited	Yes
Cost when idle	Nothing	Nothing (Fargate)	$0.10/hour per cluster + servers
Pick it when	Small APIs, event-driven work	A few container apps, small team	Company already uses Kubernetes, many services, GPUs

For many GenAI apps, **Lambda or ECS is the simpler start**. Choose EKS when Kubernetes is already your company's platform, or you need GPUs or many services together.

19 What EKS costs

Item	Price (us-east-1)
Each cluster (control plane)	$0.10 per hour (~$73 per month), even when idle
Worker nodes	Normal EC2 server prices
Auto Mode	About 12% on top of the server price
Around it	Load balancer, NAT Gateway, logs, data transfer
Old Kubernetes version	$0.60 per hour — upgrade on time

For training, use **one shared cluster** with a namespace per student, and delete lab clusters the same day.

20 What you can skip for now

These are real and useful — later. None is needed to understand the architecture.

Topic	One line
kube-proxy, VPC CNI, CoreDNS	The helper programs behind Service addresses, pod IPs and names
Service types (ClusterIP, NodePort, LoadBalancer)	Different ways a Service is exposed
Cluster endpoint access (public / private)	Who can reach the API server's address
RBAC	Fine-grained Kubernetes permissions inside the cluster
IRSA and the aws-auth ConfigMap	Older ways to do Pod Identity and access entries

Topic	One line
Volumes (EBS, EFS), StatefulSets	Disks for pods that must keep data
Resource requests and limits	How much CPU/memory each container asks for
etcd majority voting	Why masters come in 3s or 5s, in detail
Running Kubernetes yourself (kubeadm)	Building your own masters instead of using EKS
Helm, service mesh, GPU nodes, upgrades	Packaging, traffic control, model servers, yearly version changes

21 Words to know

Word	Meaning
Image / container	Packed app / running copy of it
Registry (ECR)	Where images are stored
Cluster	Control plane + worker nodes
Control plane (master)	The brain: API server, etcd, scheduler, controllers. Run by AWS in EKS
Worker node (old: slave, minion)	A server that runs pods
Pod	One running copy of your container, with one IP
Deployment	Keeps N pods running
Service / Ingress	Stable address / public web entry
Namespace	A folder for one app's objects
kubectl	The command-line tool to talk to the cluster
Desired state	What you asked for; Kubernetes keeps reality matching it

22 Key takeaways

1. A container is a running copy of an image; images are stored in ECR.
2. Kubernetes runs many containers on many servers by keeping reality equal to the desired state you write.
3. It is master–worker: the control plane (master) decides, worker nodes (old name: slaves) run the pods.
4. EKS = Kubernetes where AWS runs the control plane. Auto Mode lets AWS run the worker nodes too.
5. Every pod gets an IP from your subnet — plan subnet size for your pods.
6. Users come in through a load balancer; pods stay private.
7. Access entries: who may use the cluster. Pod Identity: what an app may call in AWS.
8. For simple GenAI apps, Lambda or ECS may be enough; pick EKS when you need Kubernetes.